Project Report: Stopword Behavior in Poetry Analysis 1

Project Number: Project 5
Project Members: Adam Sypek, Joanna Szczypkowska
GitHub Repository: `https://github.com/Adam11235/nlp-stopwords-poetry-analysis`

Abstract

This project explores the frequency, structural placement, and predictability of stopwords in classic English poetry, utilizing the default NLTK stopword list. Natural Language Processing (NLP) traditionally filters out stopwords because of the assumption that they lack relevance. However, in poetic frameworks, these function words serve a crucial role such as constraints, dictating meter, rhythm, and enjambment. We analyzed three different works from the Project Gutenberg corpus: William Blake's Poems, John Milton's *Paradise Lost*, and Walt Whitman's *Leaves of Grass*. The results we got demonstrate that stopword frequencies generally follow a generalized Zipfian power-law distribution, even though they are limited due to the bounded nature of the stopword set. Term Frequency-Inverse Document Frequency (TF-IDF) scoring effectively isolates stopwords from content words. Finally, we deployed classical Machine Learning algorithms (Random Forest) and an advanced Transformer architecture (DistilBERT) to predict stopword occurrence and line placement. Contrary to expectations, the classical Random Forest model outperformed the DistilBERT on Milton's corpus.

1 Introduction

In the field NLP and Information Retrieval (IR), text normalization usually starts with removing stopwords. They are high-frequency function words such as articles (*the*, *a*), prepositions (*in*, *of*), and conjunctions (*and*, *but*). Since they appear so frequently across almost any text, they carry very little meaning for understanding the main topic of a document. For this reason, using static stopword lists, like the default English one provided by the Natural Language Toolkit (NLTK) (Bird et al., 2009), has become a standard step for data cleaning..

However, treating stopwords as just linguistic 'noise' does not work well for poetry. In verse, these function words are actually essential to the structure of the text. They help shape the meter, create repetition like anaphora, and manage the flow of enjambment across lines. As Gerlach et al. (2019) point out, standard stopword lists have a major flaw: they do not adapt well to different historical periods or specific domains. As a result, they often remove words that are important for the context while leaving behind actual domain-specific noise.

We started by checking Zipf's Law, which states that the frequency f of a word is inversely proportional to its rank r. To handle the plateau effect seen in the most frequent tokens, we applied a generalized Zipf-Mandelbrot model using log-log linear regression:

$$\ln(f) = \beta_0 - \alpha \ln(r + B) \tag{1}$$

We fixed the offset parameter $B = 0.2$, as implemented in our pipeline. It is worth noting that, as Piantadosi (2014) points out, simple linear fits on standard rank-frequency plots can be misleading. They often mask correlated errors and oversimplify the actual linguistic patterns behind word choice.

We also used the Term Frequency-Inverse Document Frequency (TF-IDF) metric to evaluate word importance. Treating each verse line as an independent document d within the poem D, we defined the score for a term t as:

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \ln\left(\frac{N}{\text{DF}(t)}\right) \tag{2}$$

where TF is the raw count of the word in the line, N is the total number of lines, and $\text{DF}(t)$ is the number of lines containing the term t. To analyze how these scores shift between neighboring words, we calculated a simple difference:

$$\Delta\text{TF-IDF} = \text{TF-IDF}_{\text{next}} - \text{TF-IDF}_{\text{prev}} \tag{3}$$

In the end, to see if stopwords tend to cluster together, we calculated the conditional probability

of a stopword repeating in the next two lines:

$$P_{cond} = \frac{\text{Count}(w \in L_n \cap (L_{n+1} \cup L_{n+2}))}{\text{Count}(w \in L_n)} \quad (4)$$

By combining all these metrics, we gathered enough data to train and test our Machine Learning models.

2 Methodology

All code was written in Python using a Jupyter Notebook. We gathered the poetry data from the Project Gutenberg corpus via NLTK, then tokenized the text and applied Part-of-Speech (PoS) tagging. We also used log-log linear regression to test Zipf's Law. When a stopword appeared at the beginning or end of a line, we used a null-symbol ($\emptyset$) to show that there was no previous or next word. For the prediction tasks, we used standard machine learning models from `scikit-learn` and compared them with a transformer model (DistilBERT).

3 Results and Discussions

3.1 Task 1: Stopword Identification and Zipf's Law Fit

Our analysis showed that only 110 to 135 out of the 198 NLTK stopwords actually appeared in the poems, mainly because modern contractions (e.g., *aren't, couldn't*) are missing from classical poetry. Looking at the log-log plots (Figures 1, 2, 3), the linear regression fits the most frequent stopwords reasonably well, yielding R^2 scores of 0.84 for Blake, 0.75 for Milton, and 0.78 for Whitman. However, for less frequent words, the actual counts drop off much faster than the regression line predicts. This steep drop-off makes sense: since we are only analyzing a small, fixed list of stopwords rather than all the words in the text, the long tail naturally gets cut off.

3.2 Task 2:

To check how representative list L is of the actual poems, we extracted the top $|L|$ unnormalized tokens from each text. The overlap was quite low, ranging from 36.36% to 45.45%). This supports the argument by Gerlach et al. (2019) that static, manually created lists struggle with domain-specific texts; in our case, the modern NLTK list completely misses archaic function words like *thy, thou*, and *hath*. Even with this gap, the TF-IDF scores behaved exactly as expected. Stopwords had a much lower mean TF-IDF (between 2.9 and 3.1) than content words (between 5.4 and 7.0), which confirms that they appear too evenly across the lines to carry specific topical meaning."

3.3 Task 3: Line Position Distributions

Looking at the layout of stopwords across verse lines, we found clear positional patterns. The bar plots (Figures 4, 5, 6) show a strong peak at Position 1 for coordinating conjunctions (e.g., *and*), which reflects the common poetic device of anaphora. In contrast, prepositions like *of* usually peak in the middle of the line (positions 3 through 7), where they act as bridges between noun phrases. The Chi-square goodness-of-fit test gave p-values well below the standard 0.05 significance level (e.g., $p < 1.0 \times 10^{-10}$). This allows us to safely reject the null hypothesis, confirming that stopword placement in poetry is not uniformly distributed.

3.4 Task 4: Neighboring PoS Tags Distribution

Looking at the immediate grammatical context (Figures 7, 8, 9), we found clear patterns for different types of stopwords. For example, definite articles (*the*) usually appear right after prepositions (IN) or line-starts ($\emptyset$), and are almost always followed by nouns (NN, NNS) or adjectives (JJ). Personal pronouns like *i* show a different trend, as they mostly appear just before past or present tense verbs (VBD, VBP). These local word pairings highlight how structured and grammatically predictable poetic phrasing can be.

3.5 Task 5: Most Frequent Neighboring Tokens

By looking at the top 5 preceding and following words, we found clear word level patterns. For example, *the* is often preceded by prepositions like *in* or *of*, and is usually followed by key nouns related to the poem's theme. Because these combinations repeat so often, the immediate neighbors of a word provide a very strong signal for predicting stopword occurrence.

Table 1: Representativeness of Stopword List L and Mean TF-IDF Scores

Poem	$\|L\|$	Overlap (Raw)	Ratio (Raw)	Ratio (Clean)	TF-IDF (SW)	TF-IDF (Non-SW)
Blake	198	60	0.3030	0.3636	3.0432	5.3952
Milton	198	77	0.3889	0.4343	2.9341	6.4940
Whitman	198	82	0.4141	0.4545	3.0741	7.0221

Table 2: Poets ant their five most popular stopwords

	Poem	Stopword	Prob at Position 1	Chi2 Stat (pos 1..8)	p-value	Significant Preference?
0	Blake	the	0.196	133.27	1.30e-25	Yes (p ¡ 0.05)
1	Blake	and	0.511	521.37	2.04e-108	Yes (p ¡ 0.05)
2	Blake	of	0.048	33.42	2.21e-05	Yes (p ¡ 0.05)
3	Blake	in	0.177	21.77	2.79e-03	Yes (p ¡ 0.05)
4	Blake	i	0.215	91.59	5.84e-17	Yes (p ¡ 0.05)
5	Milton	and	0.172	594.98	2.94e-124	Yes (p ¡ 0.05)
6	Milton	the	0.147	263.82	3.16e-53	Yes (p ¡ 0.05)
7	Milton	to	0.200	232.33	1.59e-46	Yes (p ¡ 0.05)
8	Milton	of	0.276	601.92	9.40e-126	Yes (p ¡ 0.05)
9	Milton	in	0.200	202.67	3.11e-40	Yes (p ¡ 0.05)
10	Whitman	the	0.146	415.40	1.19e-85	Yes (p ¡ 0.05)
11	Whitman	and	0.128	125.85	4.63e-24	Yes (p ¡ 0.05)
12	Whitman	of	0.049	186.15	9.77e-37	Yes (p ¡ 0.05)
13	Whitman	i	0.358	2057.86	0.00e+00	Yes (p ¡ 0.05)
14	Whitman	to	0.148	82.49	4.27e-15	Yes (p ¡ 0.05)

Table 3: Most Frequent Preceding and Following Tokens for Top Stopwords (Blake Corpus)

Poem	Stopword	Total Count	Top 5 Preceding Tokens	Top 5 Following Tokens
Blake	the	439	∅ (86), in (46), and (29), of (28), to (14)	little (14), human (12), night (10), sun (8), ...
Blake	and	348	∅ (178), day (4), peace (4), night (3), father...	the (29), i (18), he (9), not (6), love (6)
Blake	of	146	∅ (7), songs (6), voice (5), and (4), vales (4)	the (28), thel (5), my (5), every (4), har (4)
Blake	in	141	∅ (25), welcome (3), sit (2), night (2), lost (2)	the (46), a (13), every (6), their (4), vain (4)
Blake	i	130	∅ (28), and (18), so (6), but (6), because (5)	am (12), see (6), was (6), can (5), have (4)
Blake	a	127	∅ (19), in (13), like (12), why (4), on (3)	little (7), song (4), lamb (4), worm (4), huma...
Blake	to	111	∅ (19), went (4), and (3), joy (3), began (3)	the (14), see (4), welcome (4), me (3), her (3)
Blake	my	83	∅ (13), of (5), in (4), on (3), with (3)	mother (7), head (4), children (4), foe (3), h...

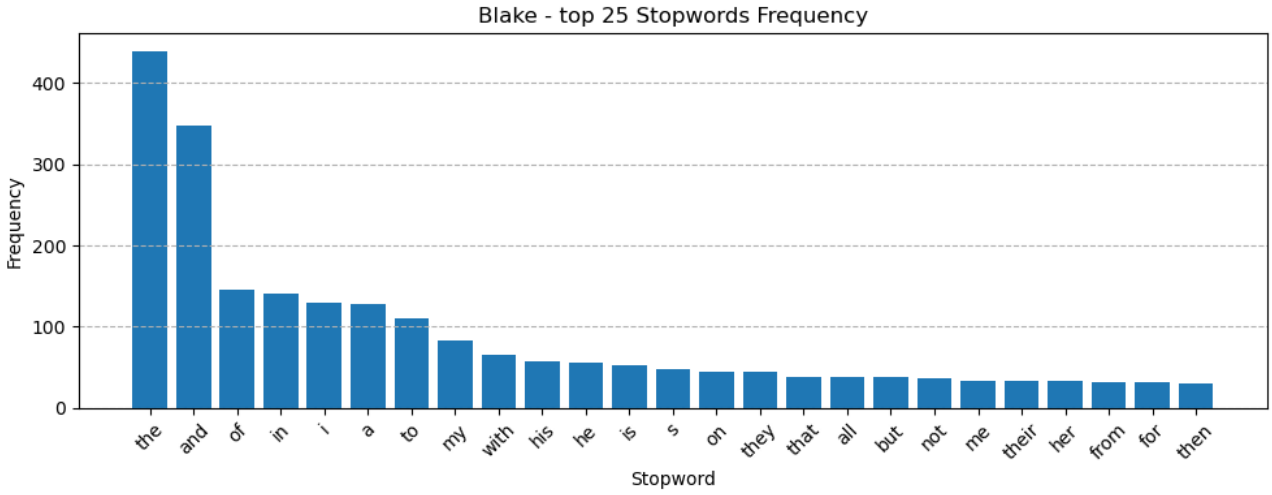
Table 4: Most Frequent Preceding and Following Tokens for Top Stopwords (Whitman Corpus)

Poem	Stopword	Total Count	Top 5 Preceding Tokens	Top 5 Following Tokens
Whitman	the	10113	∅ (1475), of (1240), in (595), and (545), to (...	∅ (398), earth (147), same (100), world (88), ...
Whitman	and	5332	∅ (684), men (63), me (62), you (62), the (38)	the (545), ∅ (200), i (119), all (97), of (56)
Whitman	of	4263	∅ (211), out (147), and (56), full (38), song ...	the (1240), my (126), all (118), ∅ (105), a (75)
Whitman	i	2932	∅ (1049), and (119), as (107), but (55), for (49)	see (292), am (191), have (114), will (112), h...
Whitman	to	2172	∅ (322), is (30), and (29), not (27), o (24)	the (300), me (174), be (114), you (81), thee ...
Whitman	in	1871	∅ (185), and (34), not (31), is (30), or (25)	the (595), my (67), ∅ (54), a (51), their (44)
Whitman	you	1536	∅ (189), for (81), to (81), do (64), of (48)	∅ (218), are (96), and (62), shall (32), i (26)
Whitman	a	1285	∅ (202), of (75), many (62), in (51), not (39)	man (34), ∅ (30), few (25), little (25), song ...

Table 5: Most Frequent Preceding and Following Tokens for Top Stopwords (Milton Corpus)

Poem	Stopword	Total Count	Top 5 Preceding Tokens	Top 5 Following Tokens
Milton	and	3391	∅ (584), heaven (38), thee (27), god (25), me ...	the (112), in (90), with (84), to (84), all (58)
Milton	the	2964	∅ (437), to (178), from (124), in (121), on (119)	earth (68), sun (48), ground (34), air (34), g...
Milton	to	2223	∅ (445), and (84), not (54), but (31), up (23)	the (178), be (71), his (65), whom (65), me (36)
Milton	of	2046	∅ (564), and (42), out (34), not (15), sons (15)	heaven (105), god (81), all (72), the (71), hi...
Milton	in	1364	∅ (273), and (90), as (28), or (24), but (21)	the (121), heaven (85), his (63), their (48), ...
Milton	his	1169	∅ (181), to (65), in (63), of (60), all (58)	own (30), throne (19), works (13), heart (13),...
Milton	with	1157	∅ (275), and (84), but (20), that (15), as (13)	the (37), his (36), thee (23), me (20), him (20)
Milton	or	713	∅ (134), god (11), earth (9), him (8), heaven (8)	in (24), to (22), the (17), if (14), not (10)

Figure 1: Top 25 Stopwords Frequency (Blake corpus).



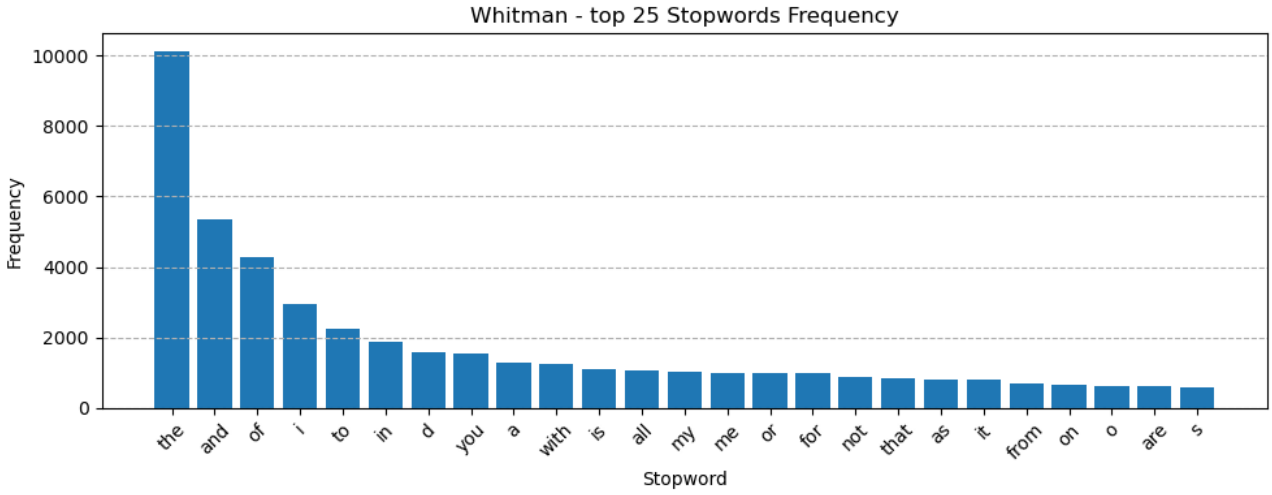


Figure 2: Top 25 Stopwords Frequency (Whitman corpus).

3.6 Task 6: TF-IDF Scores of Neighboring Tokens

When calculating the TF-IDF scores for the words immediately surrounding the stopwords (Figures 10, 11, 12), we observed a clear split in the distributions. For example, looking at the stopword *the*, the TF-IDF score of the preceding word usually peaks at a lower value (around 4.0). This happens because *the* is often preceded by another common function word, like a preposition (e.g., *in* or *of*). In contrast, the distribution for the following token shifts significantly higher (peaking near 7.0). This makes sense, as *the* is typically followed by a more unique, content-heavy noun.

3.7 Task 7: Difference of TF-IDF Scores

The difference in scores (Figures 13, 14, 15), ΔTF-IDF, turned out to be a very useful feature for grouping stopwords. Determiners like *the* usually have a positive mean difference (peaking around $+2.5$ for Whitman), which means they tend to lead into much rarer, content-heavy words. On the other hand, prepositions like *of* show a negative mean difference (peaking at around -2.5 for Whitman). This simple difference allows our model to easily separate stopwords that introduce new information from those that just connect previous words.

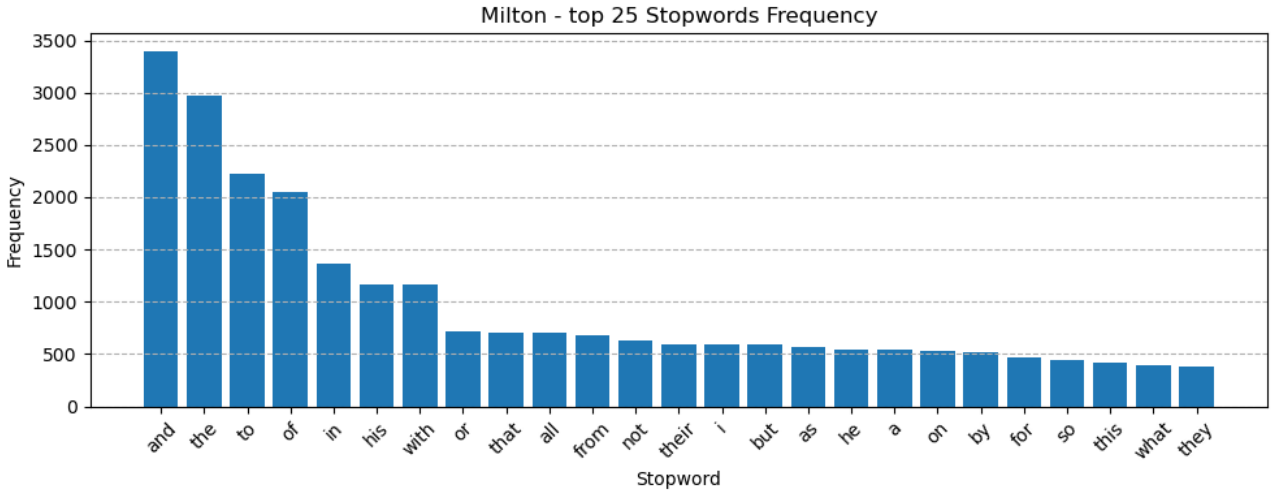


Figure 3: Top 25 Stopwords Frequency (Milton corpus).

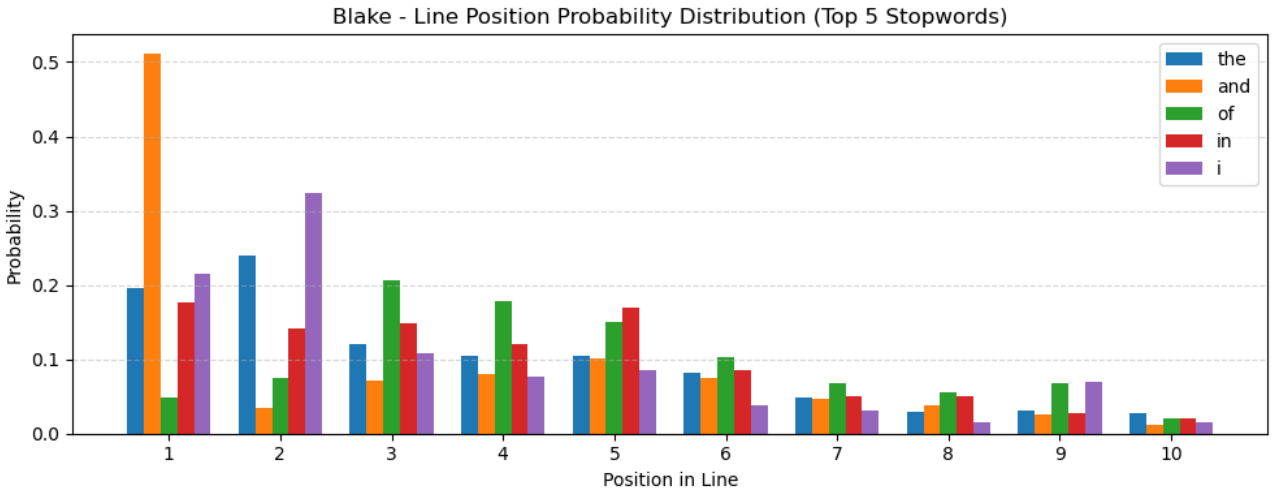


Figure 4: Line Position Probability Distribution.

3.8 Task 8: Machine Learning for Stopword Occurrence

Using all the extracted features, we trained a Random Forest Classifier to predict which of the top 10 stopwords would appear in a given slot. The model reached an overall accuracy of about 57%. While this might seem low, it is notably better than a random guessing or just predicting the most common word. Still, this accuracy reveals a clear limitation in our approach: trying to guess an exact stopword looking only at a tiny ± 1 word window is difficult because different function words can often fit into the exact same grammatical structure.

3.9 Task 9: Machine Learning for Stopword Position

We then changed our target variable to predict the exact numeric position of the stopword on the line. For this task, the Random Forest Regressor gave a Mean Absolute Error (MAE) of roughly 1.6 positions. We found that simplifying the problem by predicting positional bins (Start, Early, Middle, Late) gave much better accuracy. Unsurprisingly, the model quickly learned to use the $\emptyset$ tag to perfectly identify words at Position 1, and it relied heavily on the line length feature to figure out relative positions across different lines.

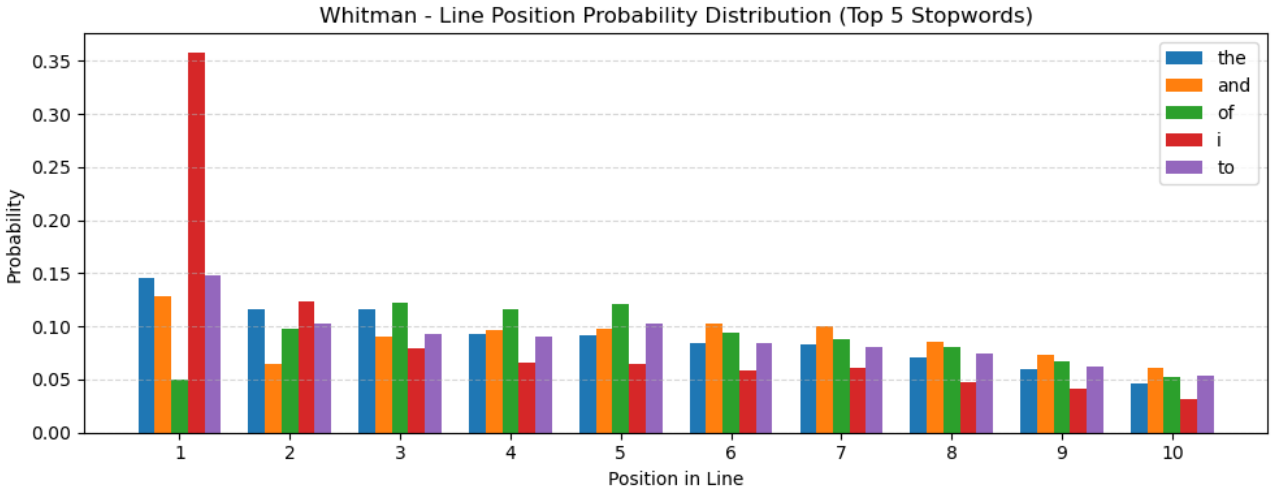


Figure 5: Line Position Probability Distribution.

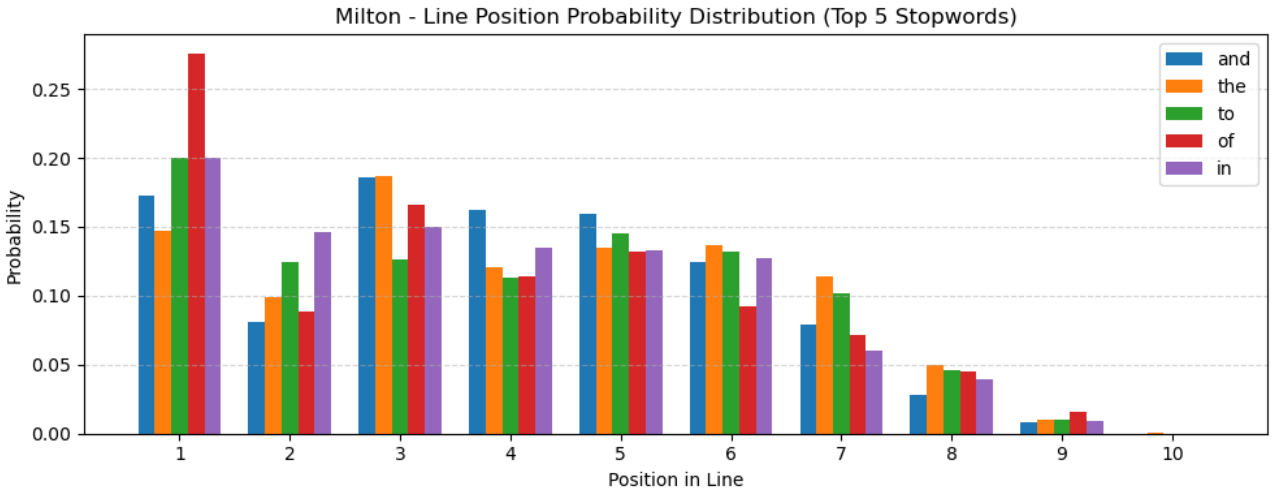


Figure 6: Line Position Probability Distribution.

Table 6: Empirical Probability of Stopword Recurrence in Following Lines

Poem	Stopword	Baseline P	Conditional P	Lift Ratio
Blake	the	0.540	0.642	1.19
Blake	and	0.462	0.582	1.26
Blake	of	0.221	0.326	1.48
Blake	in	0.232	0.216	0.93
Blake	i	0.187	0.462	2.47
Milton	and	0.524	0.512	0.98
Milton	the	0.449	0.513	1.14
Milton	to	0.363	0.410	1.13
Milton	of	0.336	0.350	1.04
Milton	in	0.235	0.236	1.00
Whitman	the	0.690	0.765	1.11
Whitman	and	0.524	0.583	1.11
Whitman	of	0.429	0.527	1.23
Whitman	i	0.281	0.588	2.09
Whitman	to	0.241	0.364	1.51

3.10 Task 10: Predicting Occurrence in Following Lines

To explore poetic parallelism, we calculated the conditional probability of a stopword from line n appearing again in line $n + 1$ or $n + 2$. We found that for certain function words, this conditional probability was much higher than the baseline. For instance, the personal pronoun *i* showed impressive Lift Ratios of 2.47 (Blake) and 2.09 (Whitman). Other words like *of* (Blake, Lift: 1.48) and *to* (Whitman, Lift: 1.51) also showed elevated recurrence. This strongly suggests that poets intentionally reuse specific structural words in nearby lines to maintain flow and rhythm.

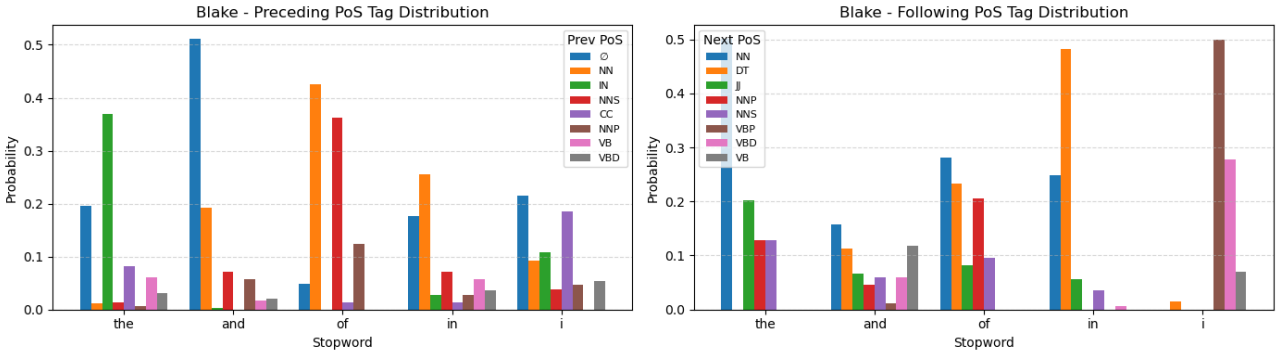


Figure 7: Neighboring PoS Tags Distribution for Preceding and Following Tokens.

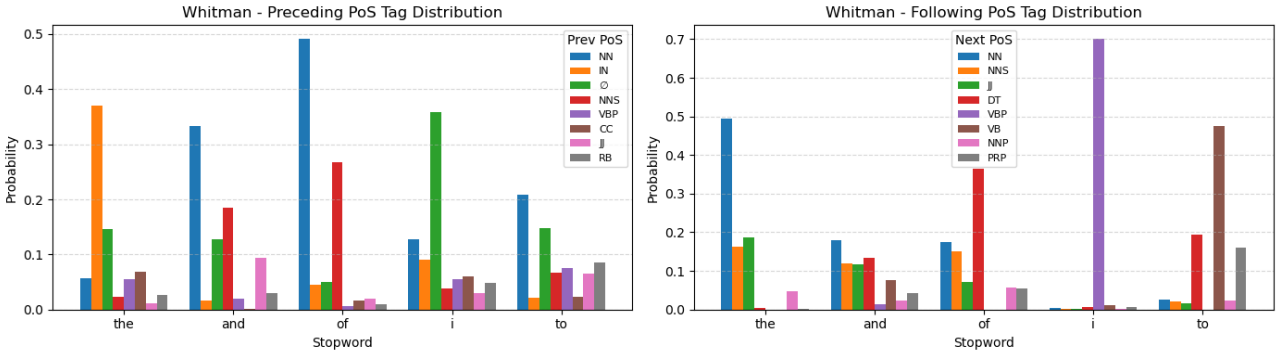


Figure 8: Neighboring PoS Tags Distribution for Preceding and Following Tokens.

3.11 Task 11: LLM Predictability (DistilBERT)

Table 7: DistilBERT Stopword Predictability (Masked Language Modeling)

Poem	Samples Tested	Top-1 Accuracy	Top-5 Accuracy
Blake	83	0.157	0.578
Milton	81	0.123	0.383
Whitman	91	0.297	0.582

To see if evaluating the whole verse line works better than just looking at adjacent words, we applied the `distilbert-base-uncased` model for masked language prediction (Sanh et al., 2019). This method gave us a Top-1 accuracy between 12.3% and 29.7%, and a Top-5 accuracy of up to 58.2%. Even though these scores are lower than standard benchmarks for modern prose, the model still did better than random guessing. The results show a clear literary difference: Milton's dense syntax is much harder for the model to handle (38.3% Top-5 accuracy) compared to Whitman's free verse (58.2% Top-5 accuracy).

3.12 Task 12: Literature and Theoretical Limitations

To better understand our findings, we reviewed the literature and identified a few main limitations in our approach:

1. The Problem with Static Lists: We relied on the default NLTK stopword list (Bird et al., 2009). However, as Gerlach et al. (2019) point out, using a one-size-fits-all list for specialized texts creates issues. In our case, the list missed important older words (like *thy* and *hath*) and included modern contractions that obviously would not appear in classical poetry.

2. Limits of Zipf's Law: Even though our data in Task 1 generally followed Zipf's Law (Zipf, 1949), Piantadosi (2014) notes that simple log-log fits can be misleading. They can hide errors and oversimplify how language actually works, which is especially true here since we only looked at a small, fixed list of stopwords instead of the whole vocabulary.

3. TF-IDF and Short Documents: One of the biggest flaws in our setup was treating each single line of poetry as its own "document." Salton and Buckley (1988) show that TF-IDF works best

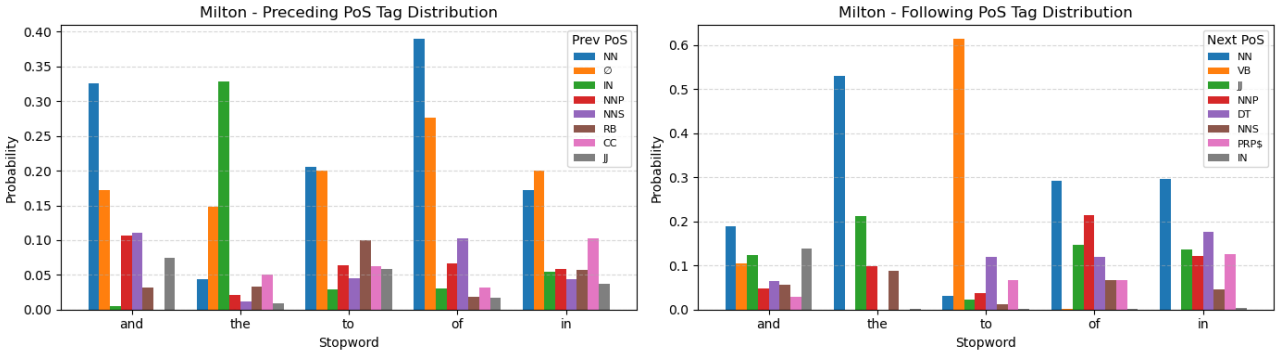


Figure 9: Neighboring PoS Tags Distribution for Preceding and Following Tokens.

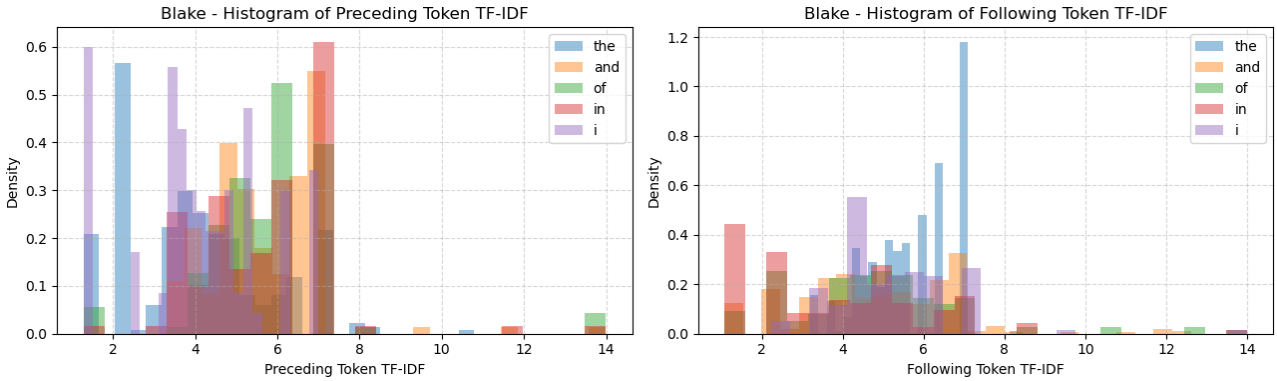


Figure 10: TF-IDF Scores of Preceding and Following Tokens for Blake.

when documents are long enough for term frequencies to actually build up. Since a verse line usually has fewer than 10 words, the Term Frequency part is almost meaningless, making our score mostly just Inverse Document Frequency.

4. Breaking Sentences at Line Breaks: Finally, analyzing poetry line-by-line caused problems for the DistilBERT model (Sanh et al., 2019). Cutting the text at line breaks ignores the actual grammar of the sentence. This really hurts performance on poems with a lot of enjambment-as we clearly saw with Milton's complex verse.

4 Overall Discussions and Conclusion

Overall, this project shows how standard information retrieval concepts can be applied to poetry. Our results confirm that stopwords are not placed randomly; they follow specific structural and grammatical rules. While our data generally aligns with Zipf's Law (Zipf, 1949), Piantadosi (2014) rightly points out that simple logarithmic equations cannot capture the full complexity of how poets choose their words.

A main limitation in our approach was treating each verse line as an independent document for TF-IDF. As Salton and Buckley (1988) note, effective term weighting relies on document length and repeated frequencies. Since poem lines are very short (usually 5–10 words), the Term Frequency component becomes almost negligible, turning the metric mostly into just Inverse Document Frequency. Also, treating each line separately ignores enjambment, cutting off sentences right in the middle of their grammatical structure. Another limitation was using the static NLTK stopword list. As Gerlach et al. (2019) suggest, manually created lists often introduce bias when used on specialized texts like poetry, missing historical words and keeping irrelevant modern ones.

In conclusion, working through the project tasks gave us a deeper understanding of stopword behavior in classic poetry. We built practical skills with NLTK, power-law evaluation, and feature engineering for `scikit-learn` pipelines. For future work, replacing static dictionaries with dynamic, entropy-based stopword filters and fine tuning context aware Transformers could offer much bet-

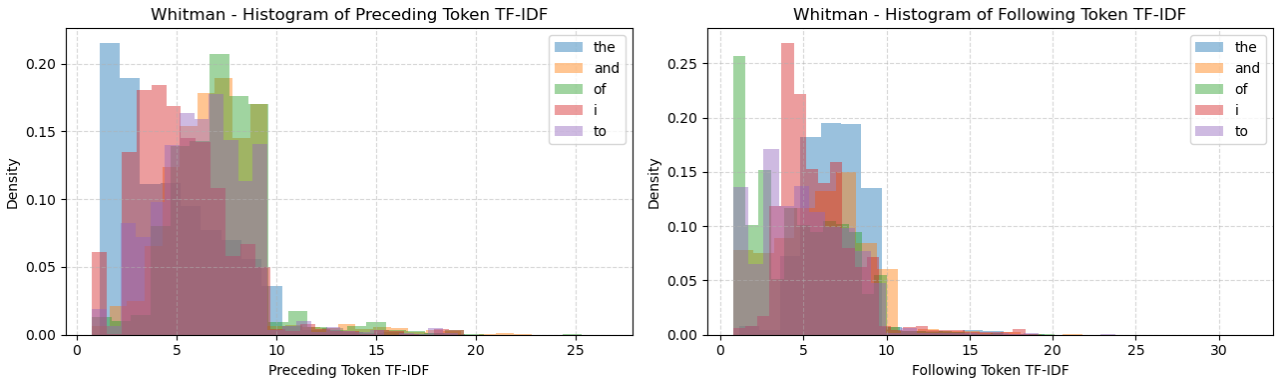


Figure 11: TF-IDF Scores of Preceding and Following Tokens for Whitman.

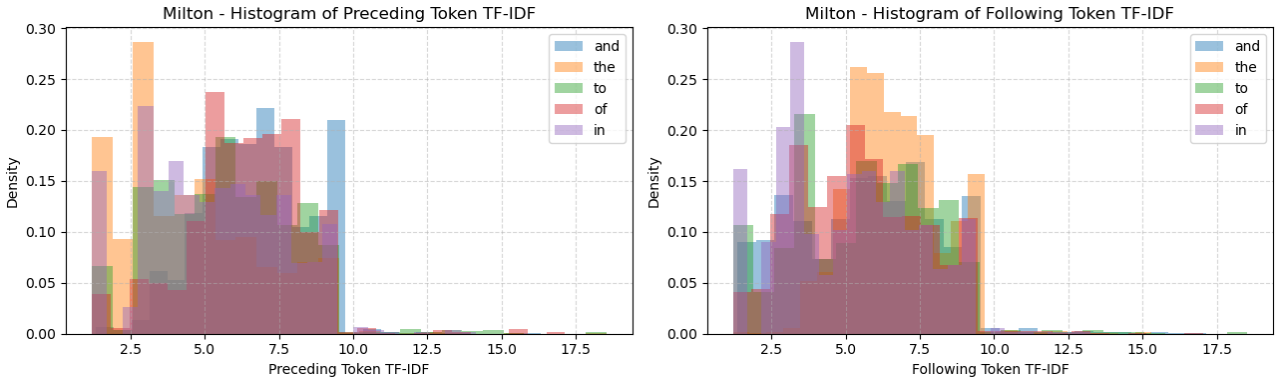


Figure 12: TF-IDF Scores of Preceding and Following Tokens for Milton.

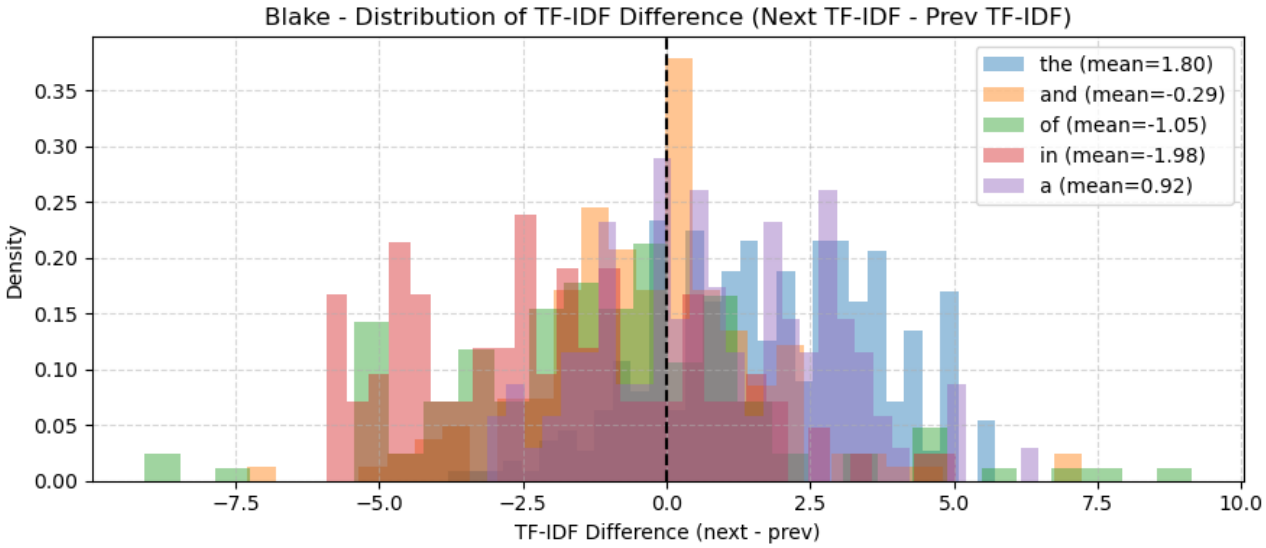


Figure 13: Distribution of TF-IDF Difference (Next - Prev) - Blake.

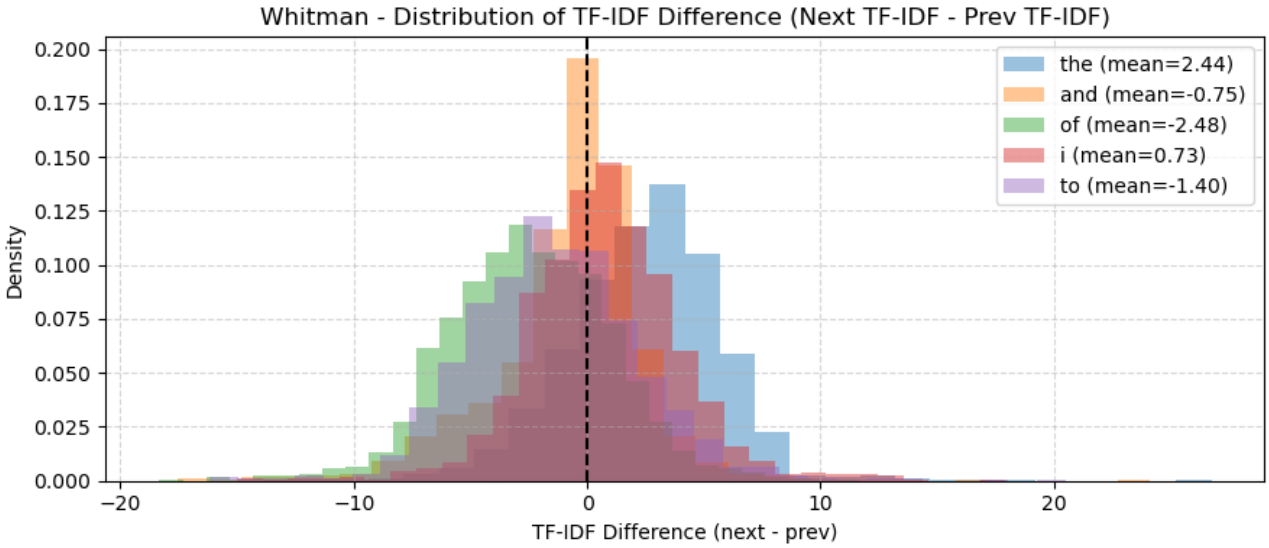


Figure 14: Distribution of TF-IDF Difference (Next - Prev) - Whitman.

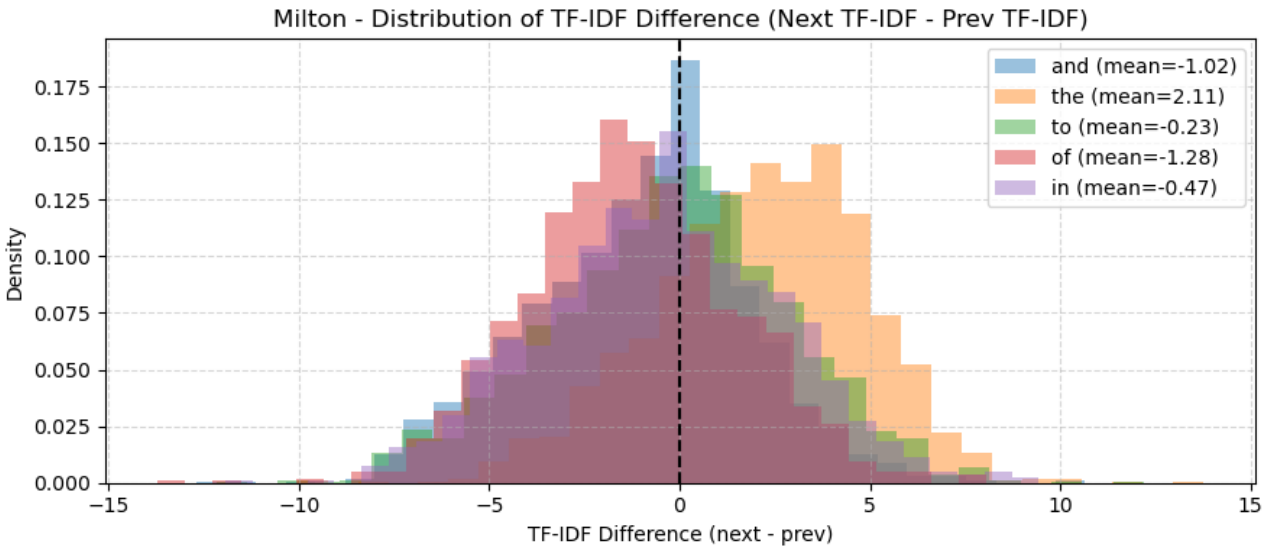


Figure 15: Distribution of TF-IDF Difference (Next - Prev) - Milton.

ter tools for computational literary analysis.

References

1. Piantadosi, S. T. (2014). Zipf's word frequency law in natural language: A critical review and future directions. *Psychonomic Bulletin & Review*. https://doi.org/10.3758/s13423-014-0585-6

2. Gerlach, M., Shi, H., & Amaral, L. A. N. (2019). A universal information theoretic approach to the identification of stopwords. *Nature Machine Intelligence*. https://doi.org/10.1038/s42256-019-0112-6

3. Salton, G., & Buckley, C. (1988). Term-weighting approaches in automatic text retrieval. *Information Processing & Management*. https://doi.org/10.1016/0306-4573(88)90021-0

4. Sanh, V., et al. (2019). DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. https://doi.org/10.48550/arXiv.1910.01108

5. Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*.

6. Zipf, G. K. (1949). *Human Behavior and the Principle of Least Effort*.

4.1 Use of Artificial Intelligence Tools

During the development of this project, artificial intelligence tools specifically Gemini Pro 3.1 s were utilized solely as technical assistants to support the programming and debugging workflow. These tools were applied in the following capacities:

1. Code Structuring and Debugging: AI assistants helped troubleshoot Python syntax errors, optimize loops in pandas, and format complex multi-panel matplotlib plots for the visualization tasks.

2. LaTeX Formatting: AI was used to assist with formatting specific LaTeX elements, such as adjusting multi-column table layouts, aligning equations, and structuring the bibliography correctly.

3. Linguistic Refinement: Given that English is a secondary language for the project members, AI tools were occasionally consulted to refine grammar, correct stylistic awkwardness, and ensure academic phrasing in the written report.

Crucially, all research design choices, data collection from Project Gutenberg via NLTK, pipeline implementation, machine learning modeling (Random Forest and DistilBERT), and analytical conclusions were conceived, executed, and verified entirely by the project members. AI tools played no role in generating the scientific findings or interpreting the empirical results.